

Experiment 3: Basic Image Handling and Processing Operations

Aim

To read an image using Python and detect its outline using the Canny edge detection function.

Algorithm

1. Import the OpenCV (cv2) library.
2. Read the input image using cv2.imread().
3. Convert the image to grayscale using cv2.cvtColor().
4. Apply the Canny edge detection function using cv2.Canny().
5. Display the original image and the outline image.
6. Wait for a key press and close all display windows.

Python Code

```
1  import sys
2  from pathlib import Path
3  import cv2
4
5
6  def detect_edges(input_path: str, show_windows: bool = False) -> None:
7      image = cv2.imread(input_path)
8
9      if image is None:
10         raise FileNotFoundError(f"Could not read image: {input_path}")
11
12         grayscale_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
13         edge_image = cv2.Canny(grayscale_image, 100, 200)
14
15         input_file = Path(input_path)
16         output_path = input_file.with_name(f"{input_file.stem}_canny{input_file.suffix}")
17         cv2.imwrite(str(output_path), edge_image)
18
19         print(f"Input image read from: {input_p (variable) output_path: Path")
20         print(f"Canny outline image saved as: {output_path}")
21
22         if show_windows:
23             cv2.imshow("Original Image", image)
24             cv2.imshow("Canny Outline Image", edge_image)
25             cv2.waitKey(0)
26             cv2.destroyAllWindows()
27
28
29  def main() -> None:
30      input_file = sys.argv[1] if len(sys.argv) > 1 else "input.jpg"
31      show_windows = "--show" in sys.argv[1:]
32      detect_edges(str(Path(input_file)), show_windows=show_windows)
33
34
35  if __name__ == "__main__":
36      main()
37
```

INPUT



OUTPUT



Result

The input image was successfully read using Python, and the outline of the image was detected using the Canny edge detection function.